EMCORE PLATFORM BACKEND

Developer Onboarding & Architecture Guide

A comprehensive manual for Software Engineers, Architecture Reviewers, and DevOps Specialists onboarding onto the EMCORE / StockOut cloud-native enterprise microservices ecosystem.

Document Version:	1.0 (Production Setup Reference)
Target Audience:	New Engineering Team Members, Developers & DevOps
Core Framework:	.NET 10.0 / ASP.NET Core 10 (C# 13+)
Architecture Pattern:	Clean Architecture, Domain-Driven Design (DDD), Microservices
Primary Database & ORM:	Microsoft SQL Server with Dapper & Stored Procedures
Messaging & Async:	MassTransit with self-hosted RabbitMQ (Outbox/Inbox pattern)
Cloud Deployment Target:	AWS ECS Fargate (UAE Region) via GitHub Actions CI/CD

■ Welcome to the EMCORE / StockOut Engineering Team!
This manual is your definitive blueprint for understanding how our backend operates. Whether you are adding a new domain feature, drafting a migration, tuning Dapper queries, or spinning up local .NET Aspire profiles, this guide ensures you build software that aligns with our enterprise reliability, scalability, and strict layering standards.

1. Executive Summary & Core Technology Stack

The **EMCORE Platform** serves as the enterprise cloud-native backend powering **StockOut** (Production Domain: api.stockout.com | Dev Domain: stockout.flowb.io). Designed to handle complex marketplace workflows—including auction bidding, escrow payments, verified grading inspections, realtime chats, and media catalogs—EMCORE is partitioned into **12 specialized business microservices**, **5 API Gateways/BFFs**, and an orchestration layer.

Platform Technical Stack

Layer / Capability	Technology Choice	Architectural Rationale
Runtime Framework	.NET 10 / ASP.NET Core 10	Flagship performance, multi-platform container efficiency, advanced OpenAPI and diagnostics support.
Architecture Pattern	Clean Architecture + DDD	Strict isolation of business Domain logic from infrastructure frameworks and external adapters.
Data Access	Dapper + SQL Server	High-speed micro-ORM using Stored Procedures exclusively for secure, database-optimized operations.
Event-Driven Messaging	MassTransit + RabbitMQ	Asynchronous pub/sub architecture utilizing transactional Inbox and Outbox patterns for reliable delivery.
Caching & State	StackExchange.Redis	Distributed in-memory caching for sub-millisecond lookups and transient state synchronization.
Search Engine	OpenSearch	Full-text search, multi-faceted filtering, and rapid listing indexing for marketplace discovery.
Object Storage	AWS S3 (Prod) / MinIO (Local)	Signed URL upload/download generation for inspection media, listing photos, and PDF invoices.
Observability & Logs	OpenTelemetry (OTEL)	Unified distributed tracing, metrics, and structured logging exported via OTLP to telemetry collectors.
Local Orchestration	.NET Aspire & Docker Compose	Allows selective multi-project debugging profiles without exhausting developer workstation CPU/RAM.

2. Repository Architecture & Clean Architecture Layering

EMCORE is structured as a **GitHub Private Monorepository**. We manage dependencies centrally using Directory.Packages.props and share compiler settings via Directory.Build.props and global.json. Every deployable microservice implements a rigid Clean Architecture directory structure.

Monorepo Directory Layout

When you clone the repository, you will navigate the following primary root folders:

```
emcore-platform/ ■■■ Emcore.Platform.slnx # Central solution referencing all services and tests ■■■ Directory.Packages.props # Central Package Management (CPM) version definitions ■■■ gateways/ # External doors: API Gateway, BFFs, MCP, Realtime Hubs ■■■ orchestration/ # .NET Aspire AppHost and ServiceDefaults ■■■ services/ # The 12 independent business microservices ■■■ building-blocks/ # Reusable technical support libraries (No domain code!) ■■■ contracts/ # Central shared contracts (OpenAPI, Events, Webhooks, MCP) ■■■ infrastructure/ # AWS ECS docs, Terraform placeholders, & local Docker setup ■■■ scripts/ # Developer utility & automation PowerShell/bash scripts ■■■ docs/ # Architecture specification specifications & onboarding guides
```

Clean Architecture Dependency Graph

In EMCORE, **dependency flow is strictly inwards**. Domain concepts never depend on technical frameworks or databases. Our continuous integration pipeline runs automated architecture tests (using NetArchTest) on every PR; violations will fail your build!

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Api & Worker Projects ■ <-- Entry points & Hosting ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ references (Composition Root only) ▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Application & Infrastructure Layers ■ <-- Handlers, Repositories, Adapters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ references ▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Domain Layer & Pure Contracts ■ <-- ZERO Infrastructure Dependencies ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Strict Architectural Rules:

• **Domain Layer:** Contains Entities, Value Objects, Enums, Domain Events, and Domain Exceptions. Must NEVER reference Dapper, EF, ASP.NET Core, or Infrastructure assemblies.

• **Application Layer:** Contains Commands, Queries, Validation, Behaviors, and Abstractions. Never depends on API or Worker projects.

• **Infrastructure Layer:** Implements Dapper IStoredProcedureExecutor repositories, MassTransit consumers, Redis caches, and S3 integrations.

• **Service Isolation:** One microservice must NEVER directly reference the assembly or database of another microservice! Communication across services is strictly via MassTransit event publishing or Gateway HTTP contracts.

3. EMCORE Microservices Portfolio

The backend is decomposed into 12 autonomous microservices. Each service owns an `.Api` project (REST endpoints, Problem Details, OpenAPI) and a `.Worker` project (MassTransit background consumers and scheduled job runners).

Service Key	Namespace	Port	Logical Database Name
identity-access	Emcore.IdentityAccess	7101	EMCORE_IDENTITY_DB
user-organization	Emcore.UserOrganization	7102	EMCORE_ORGANIZATION_DB
catalog-listing	Emcore.CatalogListing	7103	EMCORE_CATALOG_LISTING_DB
inventory-media	Emcore.InventoryMedia	7104	EMCORE_INVENTORY_MEDIA_DB
search-discovery	Emcore.SearchDiscovery	7105	EMCORE_SEARCH_DB
bidding-deal	Emcore.BiddingDeal	7106	EMCORE_BIDDING_DEAL_DB
inspection-trust	Emcore.InspectionTrust	7107	EMCORE_INSPECTION_TRUST_DB
subscription-payment	Emcore.SubscriptionPayment	7108	EMCORE_SUBSCRIPTION_PAYMENT_DB
conversation-realtime	Emcore.ConversationRealtime	7109	EMCORE_CONVERSATION_DB
notification-integration	Emcore.NotificationIntegration	7110	EMCORE_NOTIFICATION_INTEGRATION_DB
workflow-scheduler	Emcore.WorkflowScheduler	7111	EMCORE_WORKFLOW_DB
audit-reporting	Emcore.AuditReporting	7112	EMCORE_AUDIT_REPORTING_DB

Detailed Microservice Capabilities & Uses

1. Identity Access Service (Port 7101)

Namespace: Emcore.IdentityAccess | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Acts as the paramount authentication and security gatekeeper. Responsible for user credential validation, JWT token issuance, multi-factor authentication (MFA), role definitions, and dynamic permission decision evaluation. All API requests across the platform rely on security context generated by this service.

2. User Organization Service (Port 7102)

Namespace: Emcore.UserOrganization | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Manages corporate tenancies, company profiles, user organizational roles, buyer/seller verification onboarding workflows, and address/branch rosters. In a B2B marketplace, this service enables enterprise accounts to govern staff access and trading credentials.

3. Catalog Listing Service (Port 7103)

Namespace: Emcore.CatalogListing | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

The foundational bedrock of StockOut. Manages inventory product hierarchies, category attributes, lot item classifications, SKU specs, pricing metadata, listing lifecycle states (Draft, Published, Suspended, Expired), and inventory stock quantities.

4. Inventory Media Service (Port 7104)

Namespace: Emcore.InventoryMedia | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Processes all media attachments associated with listings, auctions, and inspection certificates. Communicates with AWS S3 / MinIO to issue cryptographic signed upload/download URLs, ensuring secure high-throughput image and video transfers without bottlenecking application servers.

5. Search Discovery Service (Port 7105)

Namespace: Emcore.SearchDiscovery | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Integrates deeply with OpenSearch to deliver sub-millisecond keyword search, multi-faceted filtering (price range, category, condition, location), relevance grading, and discovery feeds. Consumes listing integration events to ensure near-real-time index synchronization.

6. Bidding Deal Service (Port 7106)

Namespace: Emcore.BiddingDeal | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Implements high-frequency transactional marketplace workflows: live real-time auctions, sealed-bid mechanisms, automatic reserve price checks, negotiation room offers, deal acceptance protocols, and legal binding agreement generation between buyers and sellers.

7. Inspection Trust Service (Port 7107)

Namespace: Emcore.InspectionTrust | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Enforces marketplace credibility by managing certified third-party item grading, quality compliance inspection reports, trust scoring algorithms, vendor reliability ratings, and dispute verification evidence trails.

8. Subscription Payment Service (Port 7108)

Namespace: Emcore.SubscriptionPayment | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Governs monetization models, member tier billing cycles (Free, Gold, Enterprise), escrow payment retention for large transactions, invoice generation, and secure third-party payment provider integrations (Stripe/Bank Gateway abstraction).

9. Conversation Realtime Service (Port 7109)

Namespace: Emcore.ConversationRealtime | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Facilitates live negotiation communication between trading counterparts. Manages chat thread persistence, message encryption, read recipes, and coordinates with the Realtime Gateway for SignalR WebSocket message delivery.

10. Notification Integration Service (Port 7110)

Namespace: Emcore.NotificationIntegration | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

The multi-channel dispatch center for platform communication. Consumes domain events from RabbitMQ and formats outbox deliveries for transactional email, SMS OTPs, mobile push notifications, and external client webhook endpoints.

11. Workflow Scheduler Service (Port 7111)

Namespace: Emcore.WorkflowScheduler | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Orchestrates scheduled cron-style background jobs, stateful multi-service sagas, automated expired listing cleanups, auction closing sweeps, and retry mechanisms across the distributed architecture.

12. Audit Reporting Service (Port 7112)

Namespace: Emcore.AuditReporting | **Architecture Type:** Clean Architecture (Api + Worker + Domain + Application + Infra)

Implements tamper-resistant compliance audit trails, recording administrative overrides, financial transaction changes, security policy updates, and feeding analytics aggregators for corporate reporting dashboards.

4. Gateways & Backend-for-Frontend (BFF) Architecture

To protect internal microservices and optimize client experiences, EMCORE exposes **no internal microservice ports to public Internet networks**. All traffic flows through specialized gateways and BFFs located in the gateways/ directory.

Gateway / BFF Project	Port	Core Function & Target Consumer
Emcore.ApiGateway	7000	Built on Microsoft **YARP** (Yet Another Reverse Proxy). Acts as the primary ingress routing layer, enforcing correlation ID injection, rate limiting, SSL termination, and systemic health verifications.
Emcore.PublicBff	7010	**Backend-for-Frontend** tailored for public, unauthenticated traffic. Aggregates category trees, promotional banners, and public catalog search results into streamlined single-request mobile/web views.
Emcore.PortalBff	7020	**Backend-for-Frontend** for authenticated domain actors (Buyers, Sellers, Inspectors, Admins). Handles session security, aggregates private bidding dashboards, escrow status, and organizational analytics.
Emcore.McpGateway	7030	**Model Context Protocol (MCP)** Host Server. Exposes standardized AI tool registries and schemas, allowing trusted LLMs and AI agents (like Antigravity) to securely query and operate platform features.
Emcore.RealtimeGateway	7040	Dedicated **SignalR WebSocket Hub**. Maintains persistent real-time connections to client web/mobile apps, pushing instant auction bid out-bid alerts, live chat messages, and notification toasts.

5. Reusable Technical Building Blocks

Our modular strategy forbids duplicating utility boilerplate across services. Instead, cross-cutting technical capabilities reside in building-blocks/ as reusable NuGet-style project libraries.
CRITICAL RULE: Building blocks must contain zero domain concepts (No Listing, Bid, Deal, or Organization classes!).

Emcore.BuildingBlocks.Core

Provides core foundational paradigms: functional Result<T> wrappers, semantic exception base classes (DomainException, NotFoundException, ConflictException), deterministic Ulid ID generation, and testable IClock services.

Emcore.BuildingBlocks.Api

Implements standardized REST APIs: ASP.NET Core Problem Details error formatting, GlobalExceptionHandler, structured pagination envelopes (PagedResponse<T>, CursorResponse<T>), security headers, and Correlation ID middleware.

Emcore.BuildingBlocks.Data

The Heart of Data Access: Wraps Dapper and Microsoft.Data.SqlClient. Provides ISqlConnectionFactory and IStoredProcedureExecutor. Enforces that all database execution occurs via stored procedures with explicit command timeouts and cancellation token safety.

Emcore.BuildingBlocks.Messaging

Wraps MassTransit and RabbitMQ. Enforces the event-driven backbone via IntegrationEvent envelopes. Provides transactional **Outbox and Inbox store pattern abstractions** to guarantee zero-data-loss and idempotent message handling.

Emcore.BuildingBlocks.Security

Delivers identity evaluation: ICurrentUser, IOrganizationContext, and IPermissionChecker. Includes automatic sensitive value masking helpers to prevent PII leaks in Application diagnostics.

Emcore.BuildingBlocks.Observability

Centralizes **OpenTelemetry** metrics, traces, and activity sources. Ensures every request across HTTP, MassTransit, and Dapper generates distributed OTLP trace headers and uniform structural logging resource attributes.

Emcore.BuildingBlocks.Caching

Encapsulates distributed caching via StackExchange.Redis. Provides robust cache-key builder helpers, graceful failover capabilities, and a mockable ICacheService abstraction.

Emcore.BuildingBlocks.Storage

Object storage abstraction over AWS SDK S3 and MinIO. Handles generating secure cryptographic signed upload/download tokens (SignedUploadRequest) so clients interact directly with buckets.

Emcore.BuildingBlocks.Idempotency

Provides validation engines (IdempotencyKeyValidator) that prevent duplicate execution of high-risk operational requests (such as auction bids or recurring payments).

Emcore.BuildingBlocks.Testing

Standardized testing fixtures: WebApplicationFactory wrappers, deterministic mock clocks, in-memory configuration builders, and automated assembly scanners for architectural compliance assertion.

6. Local Development Setup & Orchestration

Running all 12 microservice APIs, 12 workers, and 5 gateways simultaneously on a single machine would cripple developer productivity. EMCORE solves this by integrating **.NET Aspire (Emcore.AppHost)** and granular **Docker Compose profiles**.

A. Infrastructure Dependencies (Docker Compose)

Located in infrastructure/docker/docker-compose.local.yml, our Compose setup leverages selective profiles so you only launch what you need.
IMPORTANT NOTE ON SQL SERVER: We intentionally **DO NOT** host SQL Server in local Docker containers! Developers connect to an established, centrally-managed Development SQL Server instance to maintain uniform stored procedure schema synchronicity.

Container Profile	Local Host Port(s)	Role in Local Environment
rabbitmq	5672 (AMQP) | 15672 (Web UI)	Message broker for MassTransit event bus integration & queue inspection.
redis	6379	In-memory database for rapid caching and session state persistence.
opensearch	9200	Local search cluster for testing index sync and marketplace faceted filtering.
minio	9000 (API) | 9001 (Web Console)	S3-compatible local object storage for testing image/media signed uploads.
otel	4317 (gRPC) | 4318 (HTTP)	OpenTelemetry Collector for capturing traces, spans, and metrics locally.

B. .NET Aspire Selective Launch Groups

Instead of hitting F5 on the entire solution, Emcore.AppHost supports targeted group execution. When debugging in Visual Studio or CLI, specify your target subsystem profile:

Aspire Group Profile	Services Launched	Use Case / Focus Area
foundation	API Gateway, BFFs, Local Infra	Gateway routing, YARP verification, AI MCP integration.
access	Identity Access, User Organization	Auth workflows, user login, JWT tokens, tenant roles.
marketplace-core	Catalog Listing, Inventory Media, Inspection Trust	Core listing lifecycle, S3 image uploads, inspection trust reports.
search	Search Discovery + OpenSearch	Search index consumption and recommendation queries.

commercial	Bidding Deal, Subscription Payment	Live auctions, bid execution, billing invoices, Stripe payment simulation.
engagement	Conversation Realtime, Notification Integration	Live chat rooms, SMS/email outbox queues, SignalR testing.
operations	Workflow Scheduler, Audit Reporting	Background cron sagas, administrative auditing trails.

C. Configuration Hierarchy & Health Checks

We enforce a standard configuration override hierarchy across all projects:
appsettings.json ➔ appsettings.{Environment}.json ➔ User Secrets (Local Only) ➔ Environment Variables ➔ AWS Secrets Manager.

Out-of-the-Box Compilation: In the Local environment, services are preconfigured with external connections (Database, Redis, Messaging) set to Enabled = false. This guarantees new engineers can compile and spin up endpoints without facing immediate SQL connection timeout failures!

Health Probing: Every service automatically registers two standardized health endpoints:
• /health/live : Liveness verification (confirms process runtime status).
• /health/ready : Readiness probe (validates active connectivity to enabled external dependencies like SQL Server and RabbitMQ). In Local mode, disabled dependencies are reported cleanly without flagging a readiness failure.

7. CI/CD Pipelines & Developer Golden Rules

Continuous Integration & Container Deployment (GitHub Actions)

Our pipeline is fully orchestrated via GitHub Actions workflows residing in .github/workflows/:

1. PR Validation (pr-validation.yml): Triggered on every pull request to main. Enforces SDK lock-file verification, formatting validation, builds in Release mode, runs Unit and Architecture tests, and executes security vulnerability scans on NuGet dependencies.

2. Main Build & Tag (main-validation.yml): Triggered upon merging to main. Repeats validation gates, detects changed deployable microservices, compiles optimized multi-stage OCI Docker container images (running under a hardened non-root user), and tags them with git commit SHAs.

3. Manual Container Build (manual-container-build.yml): Enables on-demand manual image compilation and optional deployment pushing for specific target deployable services.

■ EMCORE Engineering Golden Rules

To preserve system architecture integrity, every team member must adhere to these inviolable standards:

1. ZERO inline SQL in REST Endpoints or Workers: All relational database queries MUST flow through IStoredProcedureExecutor inside the Infrastructure layer using explicit SQL Stored Procedures.

2. NEVER leak Business Entities into Building Blocks: Emcore.BuildingBlocks.* libraries must remain totally generic technical utilities. If a code piece mentions an auction, product, or account, it belongs in a service domain!

3. Respect Microservice Boundary Isolation: Service A cannot directly invoke Service B's assemblies or access Service B's database tables. Cross-service workflows MUST be handled asynchronously via MassTransit integration event publishing or HTTP Gateway REST requests.

4. NO Secrets in Version Control: Never commit database credentials, API secrets, or encryption keys in JSON configuration files. Use Visual Studio / .NET CLI User Secrets locally (dotnet user-secrets).

5. Preserve Correlation Contexts: Always ensure OpenTelemetry distributed trace headers and Correlation IDs are carried through HTTP headers and message envelopes to enable end-to-end observability across microservice hops.

Next Steps for Onboarding Developers

1. Verify your machine matches global.json (.NET 10 SDK) and has Docker Desktop installed.
2. Open a terminal in the root directory and run dotnet build Emcore.Platform.slnx to verify zero-error compilation.
3. Run architecture and unit test suites via dotnet test.
4. Explore local infrastructure by running docker compose -f infrastructure/docker/docker-compose.local.yml --profile rabbitmq --profile redis up -d.
5. Reach out to your team lead or CODEOWNERS for access credentials to the Development SQL Server instance and AWS ECS testing environments.

EMCORE Platform Backend Ecosystem — Built with precision by the StockOut Engineering & Antigravity AI Team.